

EventZen

Event Management Platform

Product Requirements Document

Version 5.0	March 2026	Status: CURRENT	CONFIDENTIAL
Prepared by	EventZen Engineering & Product Team		
Reviewed by	CTO, VP Engineering, Lead Architects		
Document Type	Product Requirements Document (PRD)		
Stack	React + Tailwind Spring Boot Node/Express ASP.NET Core MySQL MongoDB		
Architecture	Polyglot Microservices Three-Portal Frontend Event-Driven Cloud-Native CI/CD Automated		
Supersedes	PRD v3 (June 2025) and PRD v4 (March 2026)		

Table of Contents

1. Executive Summary
 - 1.1 Strategic Objectives
 - 1.2 Technology Vision
2. System Architecture
 - 2.1 Architecture Overview
 - 2.2 Microservices Breakdown
 - 2.3 Communication Patterns
 - 2.4 Data Architecture — Polyglot Persistence
3. Product Vision & User Portals
 - 3.1 Three-Portal Model
 - 3.2 Role Model
4. Frontend Architecture
 - 4.1 Technology Stack
 - 4.2 Portal Routes & Pages
 - 4.3 Performance & CDN Strategy
5. Backend Microservices
 - 5.1 Auth & User Service
 - 5.2 Account Request Management
 - 5.3 Event Management Service
 - 5.4 Venue & Vendor Service
 - 5.5 Attendee & Ticketing Service
 - 5.6 Finance & Budget Service
 - 5.7 Notification Service
6. Caching Strategy & Performance
7. Exception Handling & Resilience
8. DevOps, CI/CD & Infrastructure
9. Security Architecture
10. Database Design (ERD Summary)
11. Feature Module Specifications
12. User Journeys
13. Non-Functional Requirements
14. Delivery Roadmap
15. Appendix

1. Executive Summary

EventZen is a next-generation, cloud-native event management platform designed to eliminate the operational bottlenecks faced by modern event planning organizations. The platform has evolved from a two-surface system into a three-portal product with distinct customer, vendor, and admin experiences, backed by a polyglot microservices architecture that scales from boutique agency operations to enterprise-grade event orchestration.

This PRD — version 5.0 — consolidates the comprehensive technical blueprint established in PRD v3 (June 2025) with all implemented product changes captured in PRD v4 (March 2026). It serves as the single authoritative reference for all engineering, product, design, QA, and DevOps stakeholders.

Problem Statement Summary

EventZen originally struggled with manual scheduling errors, siloed attendee tracking, opaque budget management, and no self-service customer portal. These inefficiencies resulted in revenue loss, client dissatisfaction, and unsustainable team overhead as event volumes grew. The current platform addresses all these gaps through automation, self-service workflows, and real-time data across all stakeholder roles.

1.1 Strategic Objectives

- Reduce event scheduling errors by 90% via automated conflict detection and workflow automation.
- Cut attendee management overhead by 75% through self-service registration and real-time dashboards.
- Achieve 100% budget traceability with real-time financial tracking and automated reporting.
- Deliver sub-200ms API response times through caching, CDN, and optimized data access layers.
- Support 10,000+ concurrent users via horizontally scalable microservices.
- Achieve 99.9% platform uptime through redundant deployment and automated failover.
- Provide distinct, role-appropriate portal experiences for customers, vendors, and administrators.

1.2 Technology Vision

The platform is built on a polyglot microservices architecture — each service independently owned, deployed, and scaled. The frontend is a React + Tailwind CSS SPA with three guarded portals (Customer, Vendor, Admin), lazy-loaded per portal. The backend comprises six microservices written in Spring Boot, Node.js/Express, and ASP.NET Core, each connected to the most appropriate database (MySQL or MongoDB). The entire platform is containerized via Docker and orchestrated with Kubernetes, with CI/CD pipelines managed through GitHub Actions.

2. System Architecture

2.1 Architecture Overview

EventZen follows a Domain-Driven Design (DDD) approach with clear bounded contexts mapped to independent microservices. Services communicate via REST APIs for synchronous operations and Apache Kafka for asynchronous, event-driven workflows. An API Gateway serves as the single entry point, handling routing, rate limiting, and authentication token validation.

High-Level Architecture (Text Diagram)
[React SPA + Tailwind CSS] — Customer Portal | Vendor Portal | Admin Portal ↓ HTTPS / WebSocket [CloudFront CDN + WAF] ↓ [API Gateway: Kong / AWS API Gateway] — Rate Limiting | JWT Validation | Routing | Load Balancing ↓ [Microservices] Auth (Spring Boot) | Event Mgmt (Spring Boot) | Venue & Vendor (Node/Express) | Attendee & Ticketing (ASP.NET) | Finance (Spring Boot) | Notification (Node/Express) ↓ Apache Kafka (Async Events) [Data Layer] MySQL (Auth, Event, Finance) | MongoDB (Venue/Vendor, Attendee/Ticketing, Notification) Redis Cache | Elasticsearch | S3 / Azure Blob

2.2 Microservices Breakdown

Service	Technology	Database	Port	Responsibility
Auth & User Service	Spring Boot 3.x	MySQL 8	:8081	JWT, RBAC, Sessions, Account Requests
Event Management Service	Spring Boot 3.x	MySQL 8	:8082	Events, Agenda, Sessions, Categories
Venue & Vendor Service	Node.js + Express	MongoDB 7	:8083	Venues, Vendors, Bookings, Contracts
Attendee & Ticketing Service	ASP.NET Core 8	MongoDB 7	:8084	Registration, Tickets, Check-in, Waitlist
Finance & Budget Service	Spring Boot 3.x	MySQL 8	:8085	Budget, Payments, Expenses, Reports
Notification Service	Node.js + Express	MongoDB 7	:8086	Email, SMS, Push, Webhooks, Kafka consumer
API Gateway	Kong / AWS API GW	Redis (config)	:8080	Routing, Auth, Rate Limit

2.3 Communication Patterns

2.3.1 Synchronous (REST)

- Client-to-Gateway: All client HTTP requests routed through the API Gateway.
- Gateway-to-Service: REST calls with JWT propagation and service discovery via Consul/Eureka.
- Inter-service REST: Used for strong-consistency queries (e.g., Finance fetching Event details).

2.3.2 Asynchronous (Apache Kafka)

Kafka topics (Avro schema-validated via Confluent Schema Registry):

- Event topics: event.created, event.updated, event.cancelled
- Booking topics: venue.booked, ticket.purchased, registration.confirmed
- Finance topics: payment.received, budget.alert, expense.logged
- Notification topics: notification.email, notification.sms, notification.push

2.4 Data Architecture — Polyglot Persistence

Service	DB Engine	Rationale	Key Collections/Tables
Auth & User	MySQL 8	ACID compliance for auth	users, roles, permissions, user_roles, account_requests
Event Management	MySQL 8	Relational event data integrity	events, event_sessions, event_agenda, categories
Venue & Vendor	MongoDB 7	Flexible schema for vendor profiles	venues, vendors, bookings, contracts
Attendee & Ticketing	MongoDB 7	High write throughput for check-in	attendees, registrations, tickets, checkins, waitlists
Finance & Budget	MySQL 8	Financial ACID transactions	budgets, payments, expenses, reports
Notifications	MongoDB 7	Schema-flexible notification logs	notifications, templates, delivery_logs, preferences, webhooks

3. Product Vision & User Portals

EventZen enables three distinct user populations through dedicated portals, each providing a tailored experience aligned to their operational responsibilities.

3.1 Three-Portal Model

Portal	Primary Users	Purpose
Customer Portal	Attendees, customers	Event discovery, checkout, tickets, registrations, account self-service, vendor access request
Vendor Portal	Vendors, organizers	Event operations, venue booking, check-in, finance, reports
Admin Portal	Platform admins	User governance, request approval, events, venues, vendors, finance, reports

3.2 Role Model

The frontend maps backend roles to portal access, with portal priority resolution as follows:

Backend Role	Portal	Priority
ADMIN	Admin Portal	1 — Highest
VENDOR or ORGANIZER	Vendor Portal	2
ATTENDEE or CUSTOMER	Customer Portal	3

4. Frontend Architecture

4.1 Technology Stack

Layer	Technology	Purpose
UI Framework	React 18 + Vite + TypeScript	Component-based SPA with fast HMR build tooling
Styling	Tailwind CSS 3.x	Utility-first design system, responsive layouts
State Management	Redux Toolkit + RTK Query	Global state, server cache, auto-invalidation
Routing	React Router v6	Code-split, lazy-loaded, guarded portal routes
Forms	React Hook Form + Zod	Type-safe form validation with Zod schemas
Charts	Recharts / Chart.js	Financial dashboards, attendance analytics
Real-time	Socket.IO / WebSocket	Live check-in counters, notifications
Testing	Vitest + React Testing Library	Unit, integration, snapshot tests
E2E Testing	Playwright	Cross-browser end-to-end test automation

4.2 Portal Routes & Pages

Public Routes:

Route	Access	Purpose
/	Public	Marketing landing page and event discovery entry
/auth	Public	Login, registration, social sign-in, recovery
/events	Public	Event listing and filtering
/events/:id	Public	Event detail, agenda, ticket selection

Authenticated Shared Routes:

Route	Access	Purpose
/events/:id/checkout/:ticketTypeId	Authenticated	Checkout and registration flow
/my/tickets	Authenticated	Ticket wallet
/my/tickets/:registrationId/pass	Authenticated	Ticket pass/QR view
/my/registrations	Authenticated	Customer registration history
/account/notifications	Authenticated	Notification inbox
/account/settings	Authenticated	Profile and account settings

Customer Portal Routes:

Route	Access	Purpose
-------	--------	---------

/customer/dashboard	Customer Portal	Customer workspace dashboard — registrations overview, spending summary, check-in status, security checklist, vendor access requests
---------------------	-----------------	--

Vendor Portal Routes:

Route	Access	Purpose
/vendor/dashboard	Vendor Portal	Vendor summary dashboard — event portfolio, venue bookings, budget summaries
/vendor/events	Vendor Portal	Event operations workspace
/vendor/venues	Vendor Portal	Venue and booking workspace
/vendor/check-in	Vendor Portal	Live event check-in operations
/vendor/finance	Vendor Portal	Budget and finance workspace
/vendor/reports	Vendor Portal	Operational and financial reporting

Admin Portal Routes:

Route	Access	Purpose
/admin/dashboard	Admin Portal	Executive and governance dashboard — user management, request queues, aggregate metrics
/admin/events	Admin Portal	Event administration
/admin/check-in	Admin Portal	Central check-in command center
/admin/venues	Admin Portal	Venue administration
/admin/finance	Admin Portal	Finance oversight
/admin/reports	Admin Portal	Analytics and reporting
/admin/vendors	Admin Portal	Vendor administration

4.3 Performance & CDN Strategy

- AWS CloudFront CDN: Static assets (JS bundles, images, fonts) served from 200+ edge locations globally.
- Code Splitting: React.lazy() + Suspense for route-level and portal-level code splitting; reduces initial bundle by ~60%.
- Asset Optimization: Vite build pipeline applies Brotli/gzip compression, image WebP conversion, SVG optimization.
- Critical Path CSS: Tailwind CSS PurgeCSS removes unused classes; production CSS < 20KB.
- Service Worker: Workbox-powered PWA caching for offline-first experience on Customer Portal.
- Preloading: Critical API calls prefetched via RTK Query's prefetchQuery on route hover.
- Lighthouse Target: Performance > 92, Accessibility > 95, Best Practices > 95, SEO > 90.

5. Backend Microservices

5.1 Auth & User Service (Spring Boot 3.x)

The Auth Service is the security backbone of the platform. It handles user lifecycle, authentication, JWT issuance, role-based access control (RBAC), and account request workflows (vendor access, deactivation, reactivation, GDPR deletion).

Core Features:

- User registration with email verification flow and email resend.
- Login via JWT (Access token: 15min, Refresh token: 7 days, stored in HttpOnly cookie).
- OAuth2 Social Login: Google Sign-In.
- Multi-Factor Authentication (TOTP via Google Authenticator): setup, verify.
- Forgot password and reset password flows.
- Role-Based Access Control: ADMIN, ORGANIZER, VENDOR, ATTENDEE roles.
- Fine-grained Permission model: module-level actions (e.g., event:create, budget:read).
- Session management with token blacklisting via Redis on logout/revoke.
- Account request lifecycle: VENDOR_ACCESS, DEACTIVATE, REACTIVATE, GDPR_DELETE.
- Admin review queue: approve/reject account requests; direct account lifecycle actions.

Auth & User API Endpoints:

Method	Endpoint	Description	Auth	Role
POST	/api/v1/auth/register	Register new user	No	Public
POST	/api/v1/auth/login	Email/password login, get JWT	No	Public
POST	/api/v1/auth/google/login	Google sign-in	No	Public
POST	/api/v1/auth/refresh	Refresh access token	RefreshToken	Public
POST	/api/v1/auth/logout	Invalidate token (Redis blacklist)	JWT	Any
GET	/api/v1/auth/me	Get current user profile	JWT	Any
PATCH	/api/v1/auth/me/profile	Update current user profile	JWT	Any
POST	/api/v1/auth/forgot-password	Request password reset	No	Public
POST	/api/v1/auth/reset-password	Reset password via token	No	Public
POST	/api/v1/auth/email-verification/resend	Resend verification email	No	Public
POST	/api/v1/auth/email-verification/confirm	Confirm email verification	No	Public
POST	/api/v1/auth/mfa/setup	Generate TOTP QR code	JWT	Any
POST	/api/v1/auth/mfa/verify	Verify TOTP token	JWT	Any
GET	/api/v1/users	List all users (paginated)	JWT	ADMIN
PUT	/api/v1/users/{id}/roles	Assign roles to user	JWT	ADMIN
DELETE	/api/v1/users/{id}	Deactivate user account	JWT	ADMIN
PATCH	/api/v1/users/{id}/reactivate	Reactivate user account	JWT	ADMIN

DELETE	/api/v1/users/{id}/gdpr/delete	GDPR delete user	JWT	ADMIN
--------	--------------------------------	------------------	-----	-------

5.2 Account Request Management

A new workflow module enabling users to submit structured lifecycle requests, visible to admins for review and action.

Supported Request Types:

- VENDOR_ACCESS — Customer requests elevation to Vendor/Organizer role.
- DEACTIVATE — User requests account deactivation.
- REACTIVATE — Deactivated user requests reactivation (also available publicly without auth).
- GDPR_DELETE — User requests permanent data erasure.

Account Request API Endpoints:

Method	Endpoint	Description	Auth
POST	/api/v1/account-requests	Submit account request	JWT
GET	/api/v1/account-requests/me	List my requests	JWT
DELETE	/api/v1/account-requests/{id}	Cancel my request	JWT
POST	/api/v1/account-requests/public/reactivation	Public reactivation request	None
GET	/api/v1/account-requests/public/reactivation/status	Public reactivation status lookup	None
GET	/api/v1/account-requests/admin	Admin request review queue	JWT/ADMIN
PATCH	/api/v1/account-requests/admin/{id}/approve	Approve account request	JWT/ADMIN
PATCH	/api/v1/account-requests/admin/{id}/reject	Reject account request	JWT/ADMIN

5.3 Event Management Service (Spring Boot 3.x)

Handles the full event lifecycle from creation to archival. Manages event categories, sessions, agendas, and integrates with the Venue service for availability checks. Publishes domain events to Kafka on every state transition.

Core Features:

- Event CRUD with rich metadata (type, category, tags, description, banner image).
- Multi-session scheduling with speaker assignment and time-conflict detection.
- Drag-and-drop agenda builder (reflected in REST API as ordered agenda items).
- Event state machine: DRAFT → PUBLISHED → REGISTRATION_OPEN → ONGOING → COMPLETED → ARCHIVED.
- Automatic capacity monitoring with threshold alerts (80%, 95%, 100%).
- Recurrence rules for periodic events (weekly, monthly).
- Full-text search powered by Elasticsearch integration.
- Category listing for frontend filters.

Supported List Filters: q, category, status, city, organizerId, page, size, from, to.

Event Service API Endpoints:

Method	Endpoint	Description	Role
POST	/api/v1/events	Create new event	ADMIN, ORGANIZER
GET	/api/v1/events	Paginated list with filters/search	Public
GET	/api/v1/events/{id}	Get event details	Public
PUT	/api/v1/events/{id}	Update event details	ADMIN, ORGANIZER
PATCH	/api/v1/events/{id}/status	Transition event state	ADMIN
DELETE	/api/v1/events/{id}	Archive event	ADMIN
POST	/api/v1/events/{id}/sessions	Add session to event	ADMIN, ORGANIZER
GET	/api/v1/events/{id}/agenda	Get ordered agenda	Public
PUT	/api/v1/events/{id}/agenda/reorder	Reorder agenda items	ADMIN, ORGANIZER
GET	/api/v1/events/search	Full-text search (Elasticsearch)	Public
GET	/api/v1/categories	List event categories	Public

5.4 Venue & Vendor Service (Node.js + Express)

Built on Node.js for its non-blocking I/O strengths in handling concurrent booking availability queries. MongoDB stores flexible venue configurations and rich vendor profiles. Now includes venue deactivation, detailed booking list, and hall-aware availability.

Core Features:

- Venue CRUD: name, address, capacity, halls, pricing, amenities, media gallery.
- Real-time availability calendar: hall-level slot-based booking conflict detection.
- Vendor catalog with service categories (catering, AV, decor, security, photography).
- Vendor rating and review system.
- Contract management: status tracking (PENDING → SIGNED → ACTIVE → COMPLETED).
- Venue-event linking with multi-hall support.
- Venue deactivation/reactivation.
- Booking confirmation notification trigger via Kafka.

Venue & Vendor API Endpoints:

Method	Endpoint	Description	Role
POST	/api/v1/venues	Register new venue	ADMIN
GET	/api/v1/venues	List venues with capacity filter	Public
GET	/api/v1/venues/{id}	Get venue details	Authenticated
PUT	/api/v1/venues/{id}	Update venue	ADMIN
DELETE	/api/v1/venues/{id}	Deactivate venue	ADMIN
GET	/api/v1/venues/{id}/availability	Check hall-aware booking availability	Authenticated
POST	/api/v1/venues/{id}/book	Book venue for event	ADMIN, ORGANIZER
GET	/api/v1/venues/bookings	List all venue bookings	ADMIN, ORGANIZER

GET	/api/v1/vendors	Vendor catalog with service filter	Authenticated
POST	/api/v1/vendors	Register vendor	ADMIN
POST	/api/v1/vendors/{id}/reviews	Submit vendor rating & review	ADMIN, ORGANIZER
POST	/api/v1/events/{id}/vendors	Hire vendor for event	ADMIN, ORGANIZER
PATCH	/api/v1/contracts/{id}/status	Update contract status	ADMIN

5.5 Attendee & Ticketing Service (ASP.NET Core 8)

ASP.NET Core provides the performance and concurrency needed for high-throughput event registration and real-time check-in. Now includes ticket type creation, my-tickets listing, CSV attendee import, and waitlist auto-promotion.

Core Features:

- Ticket type creation per event (GENERAL, VIP, SPEAKER, SPONSOR) with quantity controls.
- Attendee registration with idempotency keys (Idempotency-Key header) to prevent duplicate bookings.
- QR code generation for digital tickets (Base64 encoded, HMAC-SHA256 signed).
- Real-time check-in with QR scan validation and duplicate entry prevention.
- Waitlist management with automatic promotion on registration cancellation.
- Bulk attendee import via CSV with validation pipeline.
- My registrations and my tickets endpoints for customer portal.

Ticketing Service API Endpoints:

Method	Endpoint	Description	Role
GET	/api/v1/ticket-types?eventId={eventId}	List available ticket types for event	Public
POST	/api/v1/events/{eventId}/ticket-types	Create ticket type	ADMIN, ORGANIZER
POST	/api/v1/registrations	Register attendee (requires Idempotency-Key)	Authenticated
GET	/api/v1/registrations/me	List my registrations	Authenticated
GET	/api/v1/events/{eventId}/registrations	List event registrations	ADMIN, ORGANIZER
DELETE	/api/v1/registrations/{registrationId}	Cancel registration	Owner, ADMIN
GET	/api/v1/tickets/{ticketId}	Get ticket with signed QR code	Owner, ADMIN
GET	/api/v1/tickets/me	List my tickets	Authenticated
POST	/api/v1/checkin/scan	Validate QR and log check-in	ADMIN, STAFF
GET	/api/v1/events/{eventId}/checkin/stats	Live check-in dashboard stats	ADMIN, ORGANIZER
POST	/api/v1/events/{eventId}/waitlist	Join waitlist when sold out	Authenticated
POST	/api/v1/attendees/import	Bulk CSV import attendees	ADMIN
GET	/api/v1/health	Service health check	Public

5.6 Finance & Budget Service (Spring Boot 3.x)

Manages the complete financial lifecycle of events. Spring Boot + MySQL ensures transactional integrity. Now includes payment verification endpoint in addition to full budgeting and expense management.

Core Features:

- Budget planning: estimated, approved, and actual budget tracking.
- Line-item budgeting with category breakdown (venue, catering, AV, marketing, staff).
- Payment processing integration (Stripe / Razorpay gateway abstraction layer).
- Payment verification endpoint.
- Expense logging with receipt upload (S3 storage, URL stored in DB).
- Real-time budget variance alerts: warns at 80% and 100% utilization.
- Financial report generation per event.
- Multi-currency support with exchange rate integration.

Finance Service API Endpoints:

Method	Endpoint	Description	Role
POST	/api/v1/events/{eventId}/budget	Create event budget plan	ADMIN, ORGANIZER
GET	/api/v1/events/{eventId}/budget	Get budget with variance analysis	ADMIN, ORGANIZER
PUT	/api/v1/budgets/{budgetId}/approve	Approve budget	ADMIN
POST	/api/v1/budgets/{budgetId}/items	Add budget line item	ADMIN, ORGANIZER
POST	/api/v1/expenses	Log expense with receipt	ADMIN, ORGANIZER
POST	/api/v1/payments	Initiate payment (Stripe/Razorpay)	Authenticated
POST	/api/v1/payments/webhook	Payment gateway webhook handler	System (HMAC-verified)
POST	/api/v1/payments/verify	Verify payment status	Authenticated
GET	/api/v1/events/{eventId}/reports/financial	Financial summary report	ADMIN
GET	/api/v1/events/{eventId}/reports/financial/pdf	Export financial report as PDF	ADMIN

5.7 Notification Service (Node.js + Express)

The Notification Service is a dedicated microservice responsible for all outbound communications. In v5, the service is both event-driven (Kafka consumer) and API-driven, exposing user-facing and admin-facing REST endpoints in addition to background notification dispatch.

Design Principle: Dual-Mode Notification Service

The Notification Service was originally designed as pure event-driven (Kafka-only). As of v4/v5, it also exposes REST APIs for notification history, read state management, user preferences, push token registration, template management, and webhook subscriptions. Kafka remains the trigger mechanism for all outbound dispatch; the REST surface covers inbox management and configuration.

Core Features:

- Multi-channel dispatch: Email (AWS SES / SendGrid), SMS (Twilio / AWS SNS), Push (FCM), In-App (Socket.IO WebSocket), Webhooks (HMAC-signed HTTP callbacks).
- Handlebars HTML template engine: customizable templates stored in MongoDB with variable interpolation and version history.
- Template preview API: admin can render a template with mock data before publishing.
- User notification preferences: per-channel, per-event-type opt-in/out stored in MongoDB.
- Push token registration and removal (FCM device tokens).
- Idempotent processing: Kafka correlation IDs deduplicated via `delivery_logs`.
- Retry with exponential backoff and Dead Letter Queue (DLQ) after max retries.
- Delivery tracking: every dispatch attempt logged with provider response code and status.
- Real-time in-app notifications via Socket.IO, stored for offline delivery.
- Webhook subscriptions: external HTTP endpoints with per-subscription HMAC-SHA256 signing.
- Admin notification dashboard: delivery success rates, failure heatmap, template analytics.
- Bulk broadcast: admin-triggered announcements to all event registrants via Kafka batch.

Notification Service API Endpoints:

Method	Endpoint	Description	Auth/Role
POST	/notifications/send	Internal send endpoint (system alerts)	Service mTLS
GET	/notifications	List notification inbox (paginated)	JWT / Any
GET	/notifications/{id}	Get notification detail and delivery status	JWT / Owner
PATCH	/notifications/{id}/read	Mark notification as read/unread	JWT / Owner
DELETE	/notifications/{id}	Soft delete notification from inbox	JWT / Owner
GET	/notifications/delivery-logs	Admin delivery attempt log	JWT / ADMIN
GET	/notifications/preferences	Get user notification preferences	JWT / Any
POST	/notifications/preferences	Update notification preferences	JWT / Any
GET	/notifications/push-tokens	List push tokens	JWT / Any
POST	/notifications/push-tokens	Register push token (FCM)	JWT / Any
DELETE	/notifications/push-tokens	Remove push token	JWT / Any
GET	/notifications/templates	List notification templates	JWT / ADMIN

POST	/notifications/templates	Create notification template	JWT / ADMIN
PUT	/notifications/templates/{id}	Update template content/variables	JWT / ADMIN
POST	/notifications/templates/{id}/preview	Preview template with mock data	JWT / ADMIN
GET	/notifications/webhooks	List webhook subscriptions	JWT / ADMIN
POST	/notifications/webhooks	Create webhook subscription	JWT / ADMIN
DELETE	/notifications/webhooks/{id}	Delete webhook subscription	JWT / ADMIN
GET	/health	Service health check	Public

Kafka Topic Subscriptions (consumer group: eventzen-notification-service):

Kafka Topic	Channel(s)	Notification Triggered
user.registered	Email	Welcome email with verification link and onboarding steps
user.password.reset	Email	Password reset link with 15-minute expiry token
registration.confirmed	Email, SMS	Booking confirmation with QR-code ticket PDF attachment
registration.cancelled	Email	Cancellation confirmation with refund timeline
event.published	Email, Push	New event announcement to subscribed attendees
event.reminder.24h	Email, SMS, Push	Event reminder 24 hours before start with venue details
event.reminder.1h	SMS, Push	Final 1-hour reminder with check-in gate and QR code
event.cancelled	Email, SMS, Push	Event cancellation alert to all registered attendees
event.updated	Email, Push	Material change alert (venue, time, speaker) to registrants
waitlist.promoted	Email, SMS	Waitlist slot available with 2-hour registration window
payment.received	Email	Payment receipt with transaction ID and amount
payment.failed	Email, SMS	Payment failure alert with retry link
budget.alert.threshold	Email, In-App	Budget utilization alert at 80% and 100% to admin
vendor.contract.signed	Email	Contract countersigned confirmation to vendor and organizer
checkin.milestone	In-App	Admin alert at 50%, 75%, 90% attendee check-in milestones

Retry & Delivery Reliability:

Channel	Provider	Max Retries	Retry Strategy
Email	AWS SES / SendGrid	3	Exponential: 1 min, 5 min, 30 min
SMS	Twilio / AWS SNS	3	Exponential: 30s, 2 min, 10 min
Push (FCM)	Firebase Cloud Messaging	5	Linear: 1 min intervals; TTL 24h
In-App (WebSocket)	Socket.IO	N/A	Stored in DB; delivered on next connection

Webhook (External)	Custom HTTP callback	5	Exponential: 1, 5, 15, 30, 60 min
-----------------------	----------------------	---	-----------------------------------

MongoDB Collections — Notification Service:

Collection	Key Indexes	Schema Notes
notifications	user_id, status, created_at	Per-notification document; TTL index auto-expires read entries after 90 days
notification_templates	template_key (unique), channel	Handlebars HTML per channel; versions[] array for full rollback history
delivery_logs	notification_id, provider, sent_at	Per-attempt record with provider response code; powers retry audit trail
notification_preferences	user_id (unique)	Per-user opt-in/out per channel and event type; merged with defaults at dispatch
webhook_subscriptions	event_type, url	External webhook registrations with HMAC signing secret per subscription

6. Caching Strategy & Performance

6.1 Multi-Level Caching Architecture

Cache Layer	Technology	TTL	Cached Data
L1: Browser	HTTP Cache Headers	1h - 1 year (assets)	Static assets, API GET responses with ETag
L2: CDN	CloudFront / Cloudflare	5min - 24h	JS/CSS bundles, images, public API responses
L3: API Gateway	Kong Cache Plugin	2min	Frequently read endpoints (venue list, event list)
L4: Application	Redis 7 (Cluster mode)	Per cache key config	User sessions, JWT blacklist, event details, ticket counts
L5: DB Query	Spring Cache + Hibernate 2L	30s - 5min	Frequently queried reference data (categories, roles)

6.2 Redis Caching Patterns

Cache-Aside (Default): Service checks Redis before hitting DB. On miss: fetch from DB, write to Redis with TTL. On update/delete: invalidate affected keys immediately.

Write-Through (Ticket Counts): On ticket purchase, atomically decrement Redis counter AND write to MongoDB via Redis DECR. Redis acts as source of truth for real-time capacity.

Cache Key Strategy:

- event:{id}:details — Full event object, TTL 10min
- event:{id}:capacity — Available tickets, TTL 30s
- user:{id}:profile — User profile, TTL 5min
- venue:{id}:availability:{date} — Slot availability, TTL 2min
- finance:{eventId}:budget — Budget summary, TTL 1min

6.3 Performance Targets

Metric	Target	Cache Hit	Cache Miss
API GET (event details)	< 50ms p99	< 5ms	< 120ms
API POST (registration)	< 200ms p99	N/A	< 200ms
QR code check-in scan	< 100ms p99	< 10ms	< 100ms
Dashboard load (Admin)	< 1.5s LCP	< 800ms	< 1.5s
Concurrent users supported	10,000+	—	—
Platform uptime (SLA)	99.9%	—	—

7. Robust Exception Handling

Every service implements a standardized error handling framework ensuring consistent error responses, meaningful error codes, and complete observability across all polyglot microservices.

Unified Error Response Format

```
{ "timestamp": "2025-06-15T10:30:00Z", "status": 400, "error": "VALIDATION_ERROR", "code": "EVT-1042", "message": "Event start time must be in the future", "path": "/api/v1/events", "traceld": "abc123-xyz456", "details": [{ "field": "start_time", "issue": "Must be future date" }] }
```

7.1 Exception Handling by Service

Spring Boot (Auth, Event, Finance Services): @RestControllerAdvice with @ExceptionHandler for centralized handling. Custom exception hierarchy: EventZenException → BusinessException, SecurityException, IntegrationException. @Valid + Jakarta Bean Validation with detailed field errors. Transactional rollback on RuntimeException.

Node.js/Express (Venue & Vendor, Notification Services): Centralized error middleware as last registered middleware. express-async-errors for unhandled async rejections. Joi/Zod schema validation on all request bodies. MongoDB E11000 and CastError mapped to appropriate HTTP codes.

ASP.NET Core (Attendee & Ticketing Service): IExceptionHandler middleware with ProblemDetails RFC 9457 responses. FluentValidation pipeline. Idempotency key validation to prevent duplicate registration on network retry.

7.2 Custom Error Codes

Code	Category	HTTP Status	Description
AUTH-1001	Authentication	401 Unauthorized	Invalid or expired JWT
AUTH-1002	Authorization	403 Forbidden	Insufficient permissions
EVT-2001	Event Business Logic	409 Conflict	Venue already booked for this time slot
EVT-2002	Event Business Logic	422 Unprocessable	Event cannot transition to requested state
TKT-3001	Ticketing	409 Conflict	No tickets remaining (sold out)
TKT-3002	Ticketing	400 Bad Request	Duplicate registration (idempotency violation)
FIN-4001	Finance	422 Unprocessable	Expense exceeds approved budget
FIN-4002	Payment	402 Payment Required	Payment gateway declined transaction
SYS-9001	System	503 Service Unavailable	Downstream service circuit breaker open
SYS-9002	System	429 Too Many Requests	Rate limit exceeded at API Gateway

7.3 Circuit Breaker & Resilience

- Resilience4j (Spring Boot) and Opossum (Node.js) for circuit breaker patterns on inter-service calls.
- Circuit states: CLOSED (normal) → OPEN (failing, reject calls) → HALF_OPEN (probe recovery).
- Retry policies: exponential backoff with jitter for transient failures (max 3 retries).
- Bulkhead isolation: thread pool isolation per downstream dependency prevents cascade failures.
- Timeout configuration: 2s default on all inter-service HTTP calls.
- Fallback responses: serve stale cached data or degraded-mode responses when dependencies fail.

8. DevOps, CI/CD & Infrastructure

8.1 Containerization Strategy

Docker Build Strategy

Spring Boot services use eclipse-temurin:21-jre-alpine (~85MB). Node.js services use node:20-alpine (~50MB). ASP.NET Core uses mcr.microsoft.com/dotnet/aspnet:8.0-alpine (~80MB). All images run as non-root user (UID 1000). Multi-stage builds eliminate build toolchain from runtime images. All images tagged with Git commit SHA for traceability.

8.2 CI/CD Pipeline (GitHub Actions)

#	Stage	Actions
1	Trigger	PR open, push to main/develop, manual dispatch, release tag
2	Code Quality	ESLint (FE), Checkstyle (Java), Prettier, SonarCloud SAST, secret scanning
3	Unit Tests	JUnit 5 + Mockito (Spring), Jest (Node), xUnit (.NET), Vitest (React) — min 80% coverage gate
4	Integration Tests	Testcontainers (MySQL/MongoDB), REST-assured API tests, Playwright E2E on PR
5	Build & Package	Docker multi-stage build, tag with git SHA + semver, push to AWS ECR / GitHub Container Registry
6	Security Scan	Trivy container vulnerability scan, OWASP dependency-check, block on CRITICAL CVEs
7	Deploy Staging	Helm chart upgrade to K8s staging namespace, smoke tests, k6 performance benchmark
8	Approval Gate	GitHub Environment protection rules: required reviewer approval for production
9	Deploy Production	Blue-green deployment via Kubernetes: new version deployed to green, traffic shifted after health check
10	Post-Deploy	Slack notification, Datadog deployment marker, automated rollback if error rate > 5% in 5 minutes

8.3 Kubernetes Production Deployment

- Cluster: AWS EKS (or GKE/AKS) with managed node groups.
- Namespaces: eventzen-prod, eventzen-staging, eventzen-monitoring, eventzen-infra.
- HPA (Horizontal Pod Autoscaler): scale out based on CPU (70%) and custom Kafka consumer lag metrics.
- PodDisruptionBudget: ensures minimum 1 pod available per service during rolling updates.
- Secrets: AWS Secrets Manager / HashiCorp Vault — never stored in K8s Secrets in plaintext.
- Ingress: NGINX Ingress Controller with TLS termination (cert-manager + Let's Encrypt).
- Service Mesh: Istio for mutual TLS between services, traffic shaping, and distributed tracing.

8.4 Observability Stack

Pillar	Technology	Details
Logs	ELK Stack / OpenSearch	Structured JSON logs, Fluentbit forwarder, Kibana dashboards, 30-day retention
Metrics	Prometheus + Grafana	JVM metrics, Node.js metrics, custom business metrics (registrations/sec)
Tracing	Jaeger + OpenTelemetry	Distributed trace correlation across all services, 10% sampling, full capture on errors
Alerts	Alertmanager + PagerDuty	CPU/memory thresholds, error rate spikes, latency p99 breaches, pod restart storms
Uptime	UptimeRobot / Grafana Synthetic	External health check every 30s, public status page at status.eventzen.io

9. Security Architecture

9.1 Authentication & Authorization

- JWT Bearer tokens: RS256 asymmetric signing (private key stored in HSM/Vault).
- Access token TTL: 15 minutes; Refresh token TTL: 7 days (HTTP-only secure cookie).
- Token rotation: new refresh token issued on each access token refresh (token family pattern).
- RBAC enforcement at API Gateway level (route-level) AND service level (method-level).
- Spring Security for Java services, Passport.js for Node, ASP.NET Identity for .NET.

9.2 API Security

- HTTPS enforced everywhere; HSTS headers with 1-year max-age.
- CORS policy: strict allowlist of origins, no wildcard in production.
- Rate limiting: 100 req/min per IP for public endpoints, 1000 req/min for authenticated.
- Request size limits: 10MB max payload to prevent DoS.
- SQL injection prevention: parameterized queries (JPA/Hibernate, Spring Data).
- NoSQL injection prevention: Mongoose sanitize, input validation before query construction.
- XSS prevention: Content-Security-Policy headers, React's built-in XSS protection.
- CSRF protection: SameSite=Strict on cookies, CSRF tokens on state-changing form submissions.

9.3 Data Security

- Encryption at rest: AES-256 for DB storage (MySQL TDE, MongoDB encrypted storage).
- Encryption in transit: TLS 1.3 for all internal and external communication.
- PII handling: email and phone stored with AES-256 application-level encryption, masked in logs.
- Payment data: zero PCI scope — all card data handled by Stripe/Razorpay, tokens stored only.
- GDPR compliance: data deletion API (/api/v1/users/{id}/gdpr/delete), consent tracking.
- Audit logging: immutable audit trail for all admin actions stored in append-only log table.

10. Database Design (ERD Summary)

10.1 MySQL Schema — Auth & User Service

Table	Primary Key	Key Foreign Keys	Notable Columns
users	user_id (UUID)	—	email (unique+encrypted), password_hash, is_mfa_enabled, status
roles	role_id (UUID)	—	role_name, description
permissions	permission_id (UUID)	—	module, action, resource_type
user_roles	id (UUID)	user_id, role_id	assigned_by, assigned_at
role_permissions	id (UUID)	role_id, permission_id	—
refresh_tokens	token_id (UUID)	user_id	token_hash, expires_at, revoked, family_id
account_requests	request_id (UUID)	user_id	type (enum), status, reviewed_by, reviewed_at
audit_log	log_id (BIGINT)	user_id	action, resource_type, resource_id, ip_address

10.2 MySQL Schema — Event Management Service

Table	Primary Key	Key Foreign Keys	Notable Columns
events	event_id (UUID)	organizer_id, category_id	status (enum), banner_url, max_capacity, is_recurring
event_categories	category_id (UUID)	—	category_name, icon, color
event_sessions	session_id (UUID)	event_id	speaker_id, room, session_type, capacity
event_agenda	agenda_id (UUID)	event_id	sort_order, start_time, end_time, type
event_tags	id (UUID)	event_id	tag_name (full-text indexed)

10.3 MongoDB Collections — Venue & Vendor Service

Collection	Key Indexes	Schema Notes
venues	venue_id, city, geolocation	Embedded halls array; geospatial 2dsphere index; is_active flag
venue_bookings	venue_id + date range	Compound index on (venue_id, booking_start, booking_end) for availability queries
vendors	vendor_id, service_type, rating	Embedded service_packages array; portfolio_urls for media

event_vendors	event_id, vendor_id	Contract documents with embedded version history for audit trail
---------------	---------------------	--

10.4 MongoDB Collections — Attendee & Ticketing Service

Collection	Key Indexes	Schema Notes
attendees	attendee_id, email (unique)	Hashed email index; PII fields AES-256 encrypted at app layer
registrations	event_id + attendee_id	Unique compound index; idempotency_key field
ticket_types	event_id, type	available_quantity managed via Redis; MongoDB stores total; reconciled nightly
checkin_logs	registration_id, checkin_time	TTL index auto-expires after 2 years; gate and staff_id tracked
waitlists	event_id + position	Queue-ordered by join_time; auto-promotion triggered by Kafka event on cancellation

11. Feature Module Specifications

Module	Portal(s)	Priority	Key Capabilities
Authentication & Users	Both	P0 — Must Have	Register, login, MFA, RBAC, profile management, password reset, email verification
Account Request Management	Both	P0 — Must Have	Vendor access request, deactivate, reactivate, GDPR delete — user-submitted and admin-reviewed
Event Management	Admin / Vendor	P0 — Must Have	Event CRUD, sessions, agenda, state machine, categories
Venue Management	Admin / Vendor	P0 — Must Have	Venue CRUD, hall mgmt, booking calendar, conflict detection, deactivation
Vendor Management	Admin / Vendor	P1 — Should Have	Vendor catalog, service hiring, contracts, ratings, reviews
Attendee & Ticketing	Both	P0 — Must Have	Ticket type creation, registration, QR tickets, check-in, waitlist, CSV import
Finance & Budget	Admin / Vendor	P0 — Must Have	Budget planning, payments, verification, expense tracking, reports
Notifications	Both	P1 — Should Have	Email/SMS/push via Kafka, inbox, preferences, push tokens, templates, delivery tracking, webhooks
Analytics & Reports	Admin / Vendor	P1 — Should Have	KPI dashboards, trend analysis, CSV/PDF export
Search & Discovery	Customer	P1 — Should Have	Full-text event search, category filters, geo proximity
Customer Self-Service	Customer	P1 — Should Have	Booking history, ticket wallet, event reminders, vendor access request

12. User Journeys

12.1 Customer Journey

1. Browse events from the landing page or event listing.
2. Open event details, inspect agenda and ticket options.
3. Log in or register if required.
4. Complete checkout for a selected ticket type.
5. View registration history and ticket wallet.
6. Use ticket pass and QR code for event entry at check-in.
7. Manage notification inbox and account settings.
8. Optionally request vendor access from the customer dashboard.

12.2 Vendor Journey

1. Gain vendor/organizer access through admin role assignment or request approval.
2. Open vendor dashboard to see event portfolio and budget summaries.
3. Create and manage events; build sessions and agenda.
4. Browse venues, check hall-level availability, and create bookings.
5. Hire vendors, manage contracts, and log expenses.
6. Configure ticket types and monitor registration volume.
7. Run live check-in using the vendor check-in interface.
8. Monitor budgets, payments, and generate financial reports.

12.3 Admin Journey

1. Open admin dashboard for governance overview and aggregate metrics.
2. Review pending vendor access and system account requests.
3. Approve or reject requests; perform direct lifecycle actions (deactivate, reactivate, GDPR delete).
4. Assign or adjust user portal roles.
5. Administer events, venues, vendors, finance, and check-in via dedicated admin pages.
6. Track aggregate operational activity, ticket volume, and financial summaries.

13. Non-Functional Requirements

Category	Requirement	Target / Specification	Measurement
Performance	API Response Time	p50 < 50ms, p95 < 150ms, p99 < 300ms	Datadog APM
Performance	Page Load (LCP)	< 1.5s on 4G mobile	Lighthouse / Web Vitals
Scalability	Concurrent Users	10,000+ concurrent; HPA scales to 50 pods per service	k6 load tests
Availability	Uptime SLA	99.9% (< 8.7 hrs downtime/year)	UptimeRobot
Availability	RTO / RPO	RTO: 15 min / RPO: 1 hour (automated failover)	DR drill tests
Security	OWASP Top 10	Zero CRITICAL/HIGH findings in production	SAST + DAST scans
Security	Penetration Testing	Annual third-party pentest + quarterly automated DAST	Pentest report
Reliability	Error Rate	< 0.1% of all API requests return 5xx	Prometheus error rate alert
Maintainability	Code Coverage	> 80% unit test coverage; > 60% integration coverage	Codecov / JaCoCo
Compliance	Data Privacy	GDPR compliant: right to erasure, data portability, consent	Privacy audit
Accessibility	WCAG 2.1	AA conformance; screen reader support, keyboard navigation	axe DevTools
Access Control	Portal Route Guard	Portal routes enforce role-based access at the frontend	E2E tests
Idempotency	Ticket Registration	Registration must remain idempotent via Idempotency-Key header	Integration tests
Finance	Payment Verification	Finance operations must support payment verification and event-level reporting	Integration tests

14. Delivery Roadmap

Phase 1 — Foundation (Weeks 1–6)

Wk	Deliverable	Owner	Status
1-2	Monorepo setup, Docker Compose, CI pipeline scaffolding	Platform Team	Completed
1-2	Auth Service: register, login, JWT, RBAC, MFA, Google Sign-In	Backend - Java	Completed
2-4	Event Management Service: CRUD, state machine, categories	Backend - Java	Completed
2-4	React app skeleton: routing, portal guards, auth context, Tailwind design tokens	Frontend Team	Completed
4-6	Venue & Vendor Service (Node.js + MongoDB)	Backend - Node	Completed
4-6	Admin Portal: event list, create event, venue booking	Frontend Team	Completed

Phase 2 — Core Features (Weeks 7–12)

Wk	Deliverable	Owner	Status
7-9	Attendee & Ticketing Service (ASP.NET Core + MongoDB), ticket types, CSV import	Backend - .NET	Completed
7-9	Finance & Budget Service (Spring Boot + MySQL), payment verification	Backend - Java	Completed
9-11	QR code ticketing, check-in module, Redis caching layer	Full Stack	Completed
9-11	Customer Portal: event discovery, registration, ticket wallet	Frontend Team	Completed
11-12	Kafka integration, Notification Service (Kafka + REST APIs), end-to-end flow testing	Platform + Backend	Completed
11-12	Account Request workflows: vendor access, deactivate, reactivate, GDPR delete	Full Stack	Completed
11-12	Vendor Portal: dashboard, events, venues, check-in, finance, reports	Frontend Team	Completed

Phase 3 — Polish & Production (Weeks 13–16)

Wk	Deliverable	Owner	Status
13-14	Analytics dashboards, report PDF export, Elasticsearch full-text search	Full Stack	In Progress

13-14	Kubernetes Helm charts, staging environment, k6 load testing	Platform Team	In Progress
14-15	Security hardening: pentest, SAST, dependency audit, CSP headers	Security + Platform	Planned
15-16	UAT, bug fixes, performance optimization, documentation	All Teams	Planned
16	Production go-live: blue-green deploy, monitoring runbooks, handover	All Teams	Planned

15. Appendix

A. Technology Stack Summary

Category	Technology	Version / Notes
Frontend Framework	React + Vite + TypeScript	React 18, Vite 5, TypeScript 5.x
Styling	Tailwind CSS	v3.x, PostCSS pipeline, shadcn/ui component library
Java Backend	Spring Boot 3.x	Java 21 LTS, Spring Security 6, Spring Data JPA, Flyway
Node.js Backend	Node.js + Express 5	Node 20 LTS, TypeScript, Mongoose 8, Joi validation
.NET Backend	ASP.NET Core 8	C# 12, Entity Framework Core 8, FluentValidation, MediatR
Relational DB	MySQL 8.0	InnoDB engine, row-level locking, TDE enabled
Document DB	MongoDB 7	Replica set (3 nodes), Transactions, Atlas or self-hosted
Cache	Redis 7	Cluster mode (3 shards x 2 replicas), Redis Streams
Message Broker	Apache Kafka	Confluent Platform, Schema Registry, 3-broker cluster
Search	Elasticsearch 8	Full-text event search, geospatial queries, 3-node cluster
API Gateway	Kong Gateway 3.x	Declarative config, plugins: rate-limit, JWT, CORS, logging
CDN	AWS CloudFront	Edge caching, Lambda@Edge for auth, WAF rules
Container Runtime	Docker + Kubernetes	Docker 24+, K8s 1.29+, AWS EKS or self-managed
CI/CD	GitHub Actions	Reusable workflows, environments, OIDC for AWS auth
IaC	Terraform + Helm	Terraform for cloud infra, Helm 3 charts for K8s deployments
Monitoring	Prometheus + Grafana	kube-prometheus-stack, custom EventZen dashboards
Logging	ELK Stack	Elasticsearch + Logstash + Kibana, Fluentbit DaemonSet
Tracing	OpenTelemetry + Jaeger	Auto-instrumentation for all services, 10% sampling rate
File Storage	AWS S3 / Azure Blob	Event banners, expense receipts, ticket PDFs

B. Recommended Next Documentation Follow-Ups

- Update system design document to reflect notification REST APIs and account-request workflows.
- Update ERD to include account_requests, notification_preferences, push_tokens, and webhook_subscriptions entities.

- Add explicit API Gateway mapping documentation for all six microservices.
- Add sequence diagrams for: customer checkout, vendor onboarding, admin request approval, notification dispatch.
- Document Kafka schema registry Avro schema definitions per topic.

C. Document Revision History

Version	Date	Author	Changes
1.0 (v3)	June 2025	EventZen Engineering & Product Team	Initial PRD — full platform specification, comprehensive technical blueprint
4.0 (v4)	March 2026	EventZen Engineering & Product Team	Platform expansion: three-portal model, account requests, Google login, notification REST APIs, payment verification, venue deactivation, ticket types, my-tickets
5.0 (v5)	March 2026	EventZen Engineering & Product Team	Consolidated v3 + v4: unified reference combining full technical spec with all implemented changes; three portals, account request system, expanded API inventory, updated user journeys and roadmap status

EventZen Platform — Product Requirements Document v5.0

Version 5.0 | March 2026 | Confidential | EventZen Engineering
React + Tailwind | Spring Boot | Node/Express | ASP.NET Core | MySQL | MongoDB